Data Extraction Bot

Turning resume and invoice text into structured data

1. Introduction

Resumes and invoices often contain useful information in free-form text. Finding and retyping details by hand takes time and can lead to inconsistent records. The Data Extraction Bot is a Python API that uses a language model to read supplied text and return selected details as validated JSON.

2. Problem and goal

Different documents describe similar facts in different ways. A computer application needs those facts in predictable fields. This project accepts resume or invoice text, identifies the requested information, and arranges it into a consistent response structure.

- Extract key fields from resume text, such as name, email, skills, education, and experience.
- Extract invoice details, such as invoice number, parties, line items, tax, and total.
- Return validated JSON that another application can read.
- Avoid making up missing facts; unknown optional details are returned as null, and collections as empty lists.

3. How the system works

A client sends text to a FastAPI endpoint. The extractor provides the text and a task-specific schema to a configured language model through LangChain structured output. Pydantic checks the returned data before FastAPI sends JSON to the client.

```
Client text  ->  FastAPI endpoint  ->  LangChain extractor
    ->  Configured LLM  ->  Pydantic validation  ->  JSON response
```

4. Main API endpoints

POST /extract/resume accepts a JSON body containing text and returns ResumeData fields: name, email, phone, skills, education, experience, and summary.

POST /extract/invoice accepts a JSON body containing text and returns InvoiceData fields: invoice number/date, vendor/customer, line items, subtotal, tax, total amount, and currency.

GET /health reports that the API is running, plus the active provider and configured model. Interactive API documentation is available at /docs.

5. Example: invoice extraction

Input text:

```
Invoice INV-45. Vendor: ABC Store. Customer: Rahul.
2 chairs at $500 each. Total: $1,000.
```

Example response:

```
{
  "invoice_number": "INV-45",
  "invoice_date": null,
  "vendor_name": "ABC Store",
  "customer_name": "Rahul",
  "line_items": [{
    "description": "chairs",
    "quantity": 2,
    "unit_price": 500,
    "amount": 1000
  }],
  "subtotal": null,
  "tax": null,
  "total_amount": 1000,
  "currency": "USD"
}
```

The date, subtotal, and tax were not present in the input, so the bot left them null rather than guessing.

6. Technology

- Python and FastAPI: application and HTTP API.
- Pydantic v2: schemas and response validation.
- LangChain with structured output: connects model responses to the requested schema.
- Gemini, Ollama, or OpenAI: selectable provider, configured with environment variables.
- Pytest and HTTPX: automated tests using mocked LLM responses.

7. Reliability and safety

- Empty text is rejected; overly long input is limited by configuration.
- Temporary provider failures are retried up to three times with exponential backoff.
- Invalid model output is reported as a validation error.
- Logs avoid document contents and API keys. Provider keys belong in the local .env file, not in source control.
- Missing information is not intentionally invented by the extraction prompt.

8. Testing and observed results

Automated tests passed: 13 passed. Tests cover resume and invoice extraction with fake LLM responses, empty and oversized input, retry behavior, missing values, validation errors, provider configuration, and the health endpoint. Live Swagger checks also

returned HTTP 200 for a resume example and an invoice example.

9. Limitations

- The current API accepts plain text; it does not upload or parse PDF/image files.
- Extraction quality depends on the selected model and the clarity of the input.
- Schema validation checks the response format, but does not independently verify every fact or reconcile invoice arithmetic.
- The project is a backend API and does not currently include a user-facing web form or database.
- Cloud model providers process submitted text according to their own terms and policies; sensitive data requires careful handling.

10. Future improvements

- Add PDF and image upload with document text extraction or OCR.
- Build a simple web interface for entering text and viewing results.
- Add optional database storage, authentication, and usage limits.
- Add domain-specific checks for dates, currency, totals, and required fields.
- Add more test documents and compare extraction quality across supported models.

11. Conclusion

The Data Extraction Bot demonstrates how unstructured resume and invoice text can be converted into consistent, machine-readable JSON. It reduces repetitive manual data entry and provides a foundation that can be extended with file uploads, a user interface, and stronger business validation.